Comparative Analysis of Rocket Trajectories Under Different Planetary Gravities Using Python Simulation

Sanskriti Mishra

22-05-2026

Comparative Analysis of Rocket Trajectories Under Different Planetary Gravities Using Python Simulation

1. Abstract

This research paper presents an evaluative analysis of rocket trajectories under the gravitational conditions of Earth, Mars, and the Moon using Python-based simulations.

The study evaluates changes in flight time, maximum height, and horizontal range under various gravity accelerations using projectile motion equations. To compare planetary effects and display rocket trajectories, a Python simulation using NumPy and Matplotlib was created. The findings underscore the importance of computational models in aeronautical engineering and show how gravity substantially affects projectile velocities.

2. Introduction

Rocket trajectory modeling plays an important role in aerospace engineering, physics, and computational science. The projection of a rocket's motion during flight helps engineers and researchers analyze how factors such as gravity, launch angle, and initial velocity affect the overall trajectory of the rocket. With the enhancement of computational technologies, simulations have become an essential tool for analyzing and visualizing projectile motion more efficiently and accurately.

Computational simulations are widely used in modern engineering because they allow complex physical systems to be studied using mathematical models and programming techniques. In aerospace engineering, trajectory simulations are commonly used to estimate flight paths, analyze motion under varying environmental conditions, and improve the understanding of rocket dynamics. These simulations also provide students and researchers with practical exposure to scientific problem-solving and numerical analysis.

This research paper presents a comparative computational analysis of rocket trajectories under different planetary gravitational conditions using Python-based simulation techniques. The study focuses on analyzing projectile motion on Earth, the Moon, and Mars by applying fundamental equations of motion and visualizing the resulting trajectories through graphical simulation. The simulation was developed using Python programming libraries, including NumPy and Matplotlib, to calculate and animate the trajectory of the rocket under varying gravitational accelerations.

The central aim of this study is to investigate how changes in gravitational force influence important flight parameters such as maximum height, horizontal range, and time of flight. In addition, the paper aims to demonstrate the usefulness of computational simulations in understanding aerospace concepts and developing practical programming applications related to engineering and physics.

This study further underscores the role of computational problem-solving in aerospace engineering and encourages the incorporation of programming and simulation technologies into scientific learning and research.

3. Physics Background

3.1 Projectile motion is a foundational concept in classical physics and aerospace engineering. It refers to the motion of an object projected into the air under the impact of gravity. The object's trajectory is generally parabolic when air resistance is neglected. The study of projectile motion has played a significant role in the historical development of mechanics and motion analysis.

The foundations of projectile motion were extensively developed according to the findings of Galileo Galilei during the late sixteenth and early seventeenth centuries. Galileo’s investigations demonstrated that horizontal and vertical motions are independent of one another, leading to a deeper understanding of how projectiles move under gravitational force. His work established important theoretical principles that later became fundamental to classical mechanics and engineering physics.

Projectile motion can be analyzed by separating the initial launch velocity into horizontal and vertical components. The horizontal motion remains constant in the

absence of external forces, while the vertical motion is continuously affected by gravitational acceleration. These principles form the basis of trajectory prediction and are widely applied in aerospace engineering, missile guidance systems, and computational simulations.

In modern engineering and scientific research, projectile motion is commonly studied using computational modeling techniques. Simulations allow engineers and researchers to visualize trajectories, analyze motion under different environmental conditions, and better understand the effects of gravity and velocity on flight dynamics.

3.2 The computational simulation developed in this study is based on the fundamental equations of projectile motion. These equations are used to calculate the trajectory, velocity components, time of flight, maximum height, and horizontal range of the rocket under different gravitational conditions.

Velocity Components

The initial launch velocity is resolved into horizontal and vertical components:

Horizontal Velocity:

$$v_x = v \cos \theta$$

Vertical Velocity:

$$v_y = v \sin \theta$$

where:

- v is the initial launch velocity,
- θ is the launch angle.

Horizontal Displacement:

The horizontal distance covered by the projectile is calculated by

$$x = v_x t$$

where:

- x represents horizontal displacement,
- v_x represents horizontal velocity,
- t represents time.

Vertical Displacement:

The vertical distance covered by a projectile is calculated by

$$y = v_y t - \frac{1}{2} g t^2$$

where:

- y represents vertical displacement,
- g represents gravitational acceleration.

Time of Flight

The total time of flight for which a projectile remains in motion is calculated by

$$T = \frac{2v_v}{g}$$

where:

- T represents the time of flight.

Maximum Height:

The maximum height covered by a projectile during its time of flight is calculated by

$$H = \frac{2v_y}{g}$$

Where:

- H is the maximum height.

Horizontal Range:

The total horizontal distance travelled by a projectile is calculated by

$$R = v_x T$$

where:

- R is the horizontal range

- T represents the total time of flight

4. Methodology

The purpose of this project was to create a Python-based rocket trajectory simulator capable of modeling projectile motion under different gravitational conditions. The simulator was developed to demonstrate how programming and physics can be combined to create computational simulations. The project was designed using beginner-level aerospace and programming concepts while focusing on accuracy, visualization, and user interaction.

The Python programming language was used for the entire project because of its simplicity, readability, and strong scientific computing libraries. The main libraries used in this project were NumPy and Matplotlib. NumPy was used for mathematical calculations and handling numerical data, while Matplotlib was used to generate graphs and create animated visualizations of the rocket trajectory. [1][2]

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
```

The program begins by collecting user inputs using Python's `input()` function. The user is asked to enter the launch velocity and launch angle of the rocket. Conditional statements (`if, elseif,` and else) were then used to allow the user to select among different planetary environments, such as Earth, the Moon, and Mars. Based on the selected option, the program assigns different gravitational values to simulate how gravity affects projectile motion on different planets.

```
velocity = float(input("Enter launch velocity (m/s): "))
angle = float(input("Enter launch angle (degrees): "))

print("\nChoose Planet:")
print("1. Earth")
print("2. Moon")
print("3. Mars")
```

After collecting the inputs, the launch angle was converted from degrees to radians using NumPy's `radians()` function because trigonometric functions in Python operate in radians. The horizontal and vertical velocity components were then calculated using trigonometric functions such as cosine and sine.

```
theta = np.radians(angle)
```

The simulator uses projectile motion equations to calculate the rocket's horizontal and vertical positions over time. NumPy's `linspace()` function was used to generate multiple time intervals between launch and landing, allowing smooth trajectory calculations. Arrays were used to store the values of horizontal distance (`x`) and vertical height (`y`) during the rocket's flight.

```
t = np.linspace(0, time_of_flight, 500)
```

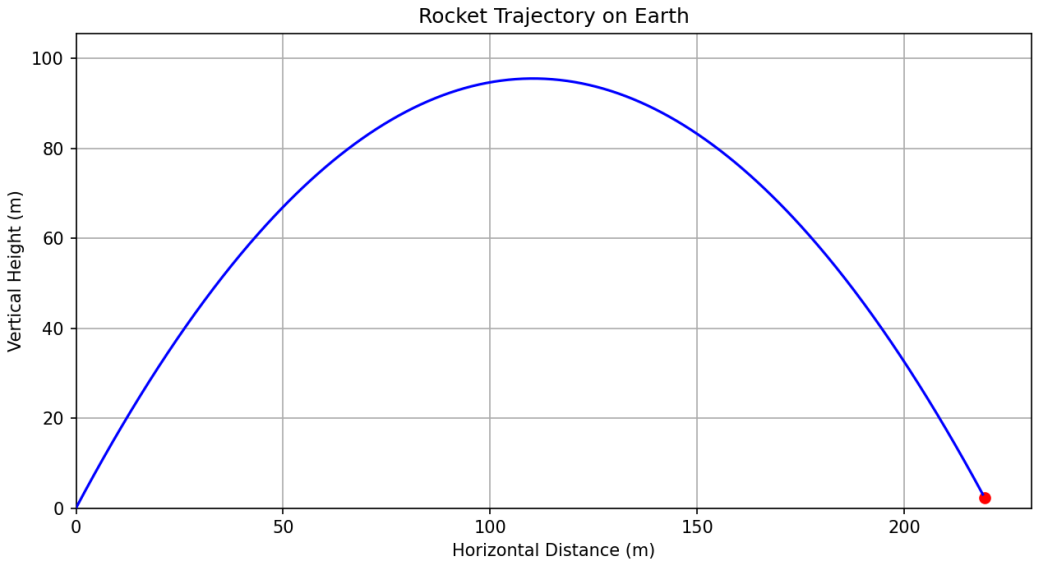
The program also calculates important flight parameters such as:

- Time of flight
- Maximum height
- Horizontal range

These values were displayed using formatted output statements (print() and formatted string literals) to improve the readability and presentation of the results.

```
print(f"Planet: {planet}")
print(f"Gravity: {gravity} m/s²")
print(f"Time of Flight: {time_of_flight:.2f} seconds")
print(f"Maximum Height: {max_height:.2f} meters")
print(f"Horizontal Range: {range_distance:.2f} meters")
```

For visualization, Matplotlib was used to generate a trajectory graph showing the rocket's motion. Labels, titles, axis limits, and grid lines were added to improve graph clarity and make the simulation easier to understand.



An animation feature was also implemented using Matplotlib's `FuncAnimation()` function. Functions such as `init()` and `update()` were created to update the rocket's position during the simulation continuously. The `update()` function controls the rocket's movement frame by frame while simultaneously displaying the trajectory path traveled by the rocket. This allowed the simulator to visually represent projectile motion dynamically instead of only displaying static graphs.

```
fig, ax = plt.subplots(figsize=(10, 5))

# Axis limits
ax.set_xlim(0, max(x) + 10)
ax.set_ylim(0, max(y) + 10)

# Labels and title
ax.set_title(f"Rocket Trajectory on {planet}")
ax.set_xlabel("Horizontal Distance (m)")
ax.set_ylabel("Vertical Height (m)")

ax.grid(True)
```

```
# Rocket point
rocket, = ax.plot([], [], 'ro')

# Trajectory line
trajectory, = ax.plot([], [], 'b-')
```

```
def init():
    rocket.set_data([], [])
    trajectory.set_data([], [])
    return rocket, trajectory

# Animation function
def update(frame):

    # Rocket position
    rocket.set_data([x[frame]], [y[frame]])

    # Path traveled so far
    trajectory.set_data(x[:frame], y[:frame])

    return rocket, trajectory
```

```
ani = FuncAnimation(
    fig,
    update,
    frames=len(t),
    init_func=init,
    interval=20,
    blit=True
)

plt.show()
```

The overall structure of the program was divided into smaller sections using comments to improve code organization and readability. Variables were used to store user inputs, calculations, and trajectory data throughout the simulation process.

The methodology used in this project demonstrates how Python programming can be applied to simulate basic aerospace and projectile motion concepts. By combining mathematical calculations, conditional logic, arrays, functions, graph plotting, and animation techniques, the simulator was able to model rocket trajectories under different gravitational conditions effectively.

This project also helped improve understanding of computational problem-solving, logical programming structure, scientific visualization, and the practical application of physics through software development. Additionally, the use of visualization and animation made the simulation more interactive and easier to analyze.

[1] NumPy Documentation. Available at: https://numpy.org/doc/

[2] Matplotlib Documentation. Available at: https://matplotlib.org/stable/index.html

5. Planetary Gravity Comparison and Analysis

Planetary Gravity Table

Planet	Gravity(m/s^2)
Earth	9.81
Moon	1.62
Mars	3.71

Analysis

The simulator was designed to compare projectile motion under different gravitational conditions on Earth, the Moon, and Mars. By changing the gravitational value in the program, noticeable differences were observed in the rocket's trajectory, flight time, and maximum height.

On the Moon, the rocket remained in the air for the longest duration because of the Moon's very low gravitational force (1.62 m/s²). The lower gravity caused the rocket to descend more slowly, resulting in a higher and longer trajectory compared to the other planets.

On Mars, the trajectory was moderate because Mars has a gravitational force stronger than the Moon but weaker than Earth. The rocket traveled for a reasonable amount of time and achieved a balanced trajectory shape between the two extremes.

On Earth, the rocket experienced the steepest descent due to Earth's higher gravitational force (9.81 m/s^2). The stronger gravity pulled the rocket downward more quickly, resulting in a shorter flight time and a comparatively lower trajectory.

These comparisons helped demonstrate how gravity directly affects projectile motion and trajectory behavior. The analysis also showed how computational simulations can be used to visualize and compare physical conditions in different planetary environments effectively.

6. Results and Observations

The rocket trajectory simulator successfully demonstrated how gravitational force and launch angle affect projectile motion under different planetary conditions. By changing the launch angle, velocity, and gravitational values, noticeable differences were observed in the rocket's flight path, maximum height, horizontal range, and time of flight.

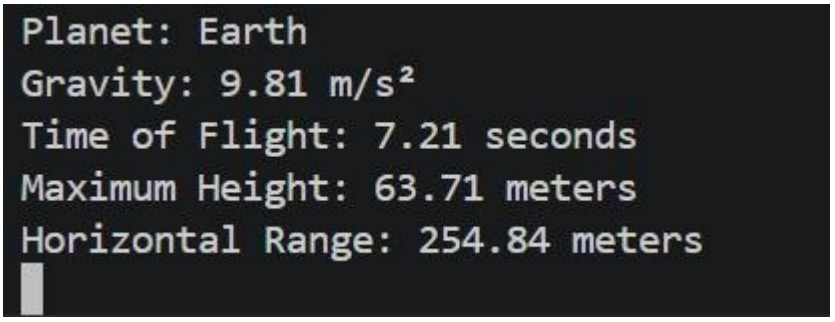
One of the most important observations from the simulation was that lower gravity increased the total time of flight of the rocket. On the Moon, where gravity is significantly lower than on Earth, the rocket stayed in the air for a much longer duration and followed a wider trajectory. In contrast, Earth's stronger gravitational force caused the rocket to descend more quickly, resulting in a shorter flight time and steeper trajectory.

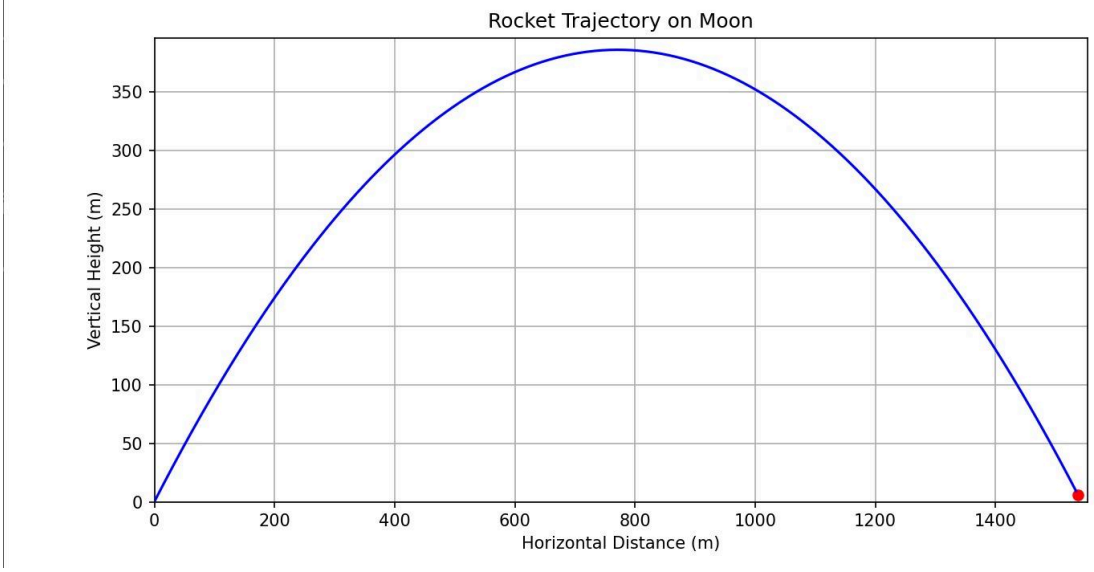
The simulation also showed that the maximum height increased when the gravitational force decreased. Since the Moon has the lowest gravity among the selected planetary environments, the rocket achieved the highest vertical displacement there. Mars's graphical trajectory plots and animation outputs helped visualize these observations more clearly. The animated simulation made it easier to compare projectile motion on different planets and understand how changes in gravity directly influence rocket movement.

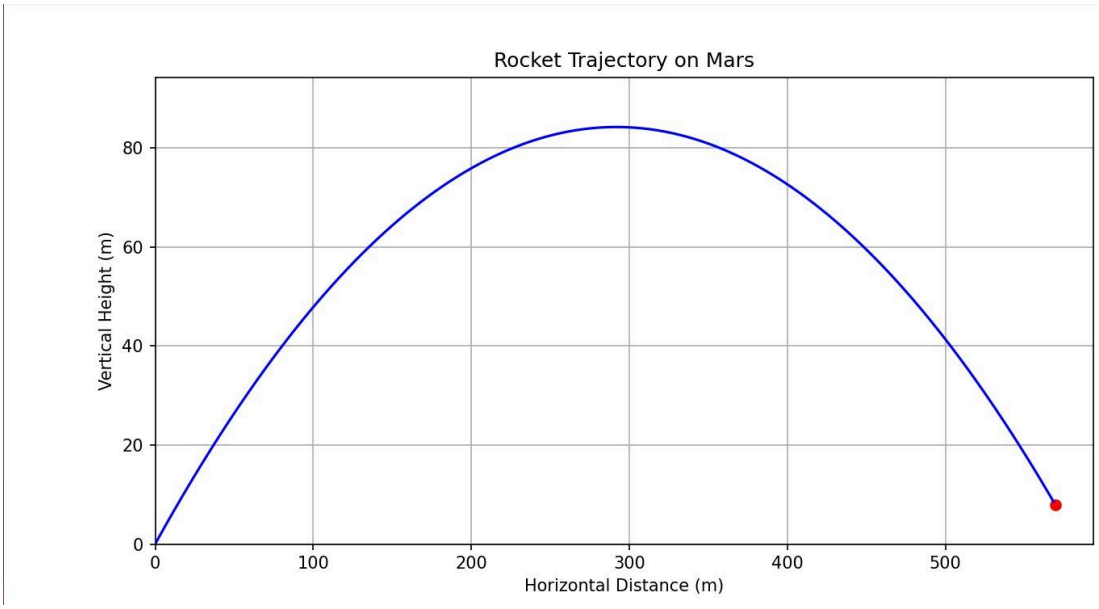
The results obtained from this project demonstrate how computational simulations can effectively represent real-world physics concepts using programming and mathematical modeling. Additionally, the project highlighted the practical application of Python programming in scientific visualization, simulation systems, and computational analysis. owed moderate results because its gravity lies between Earth and the Moon.

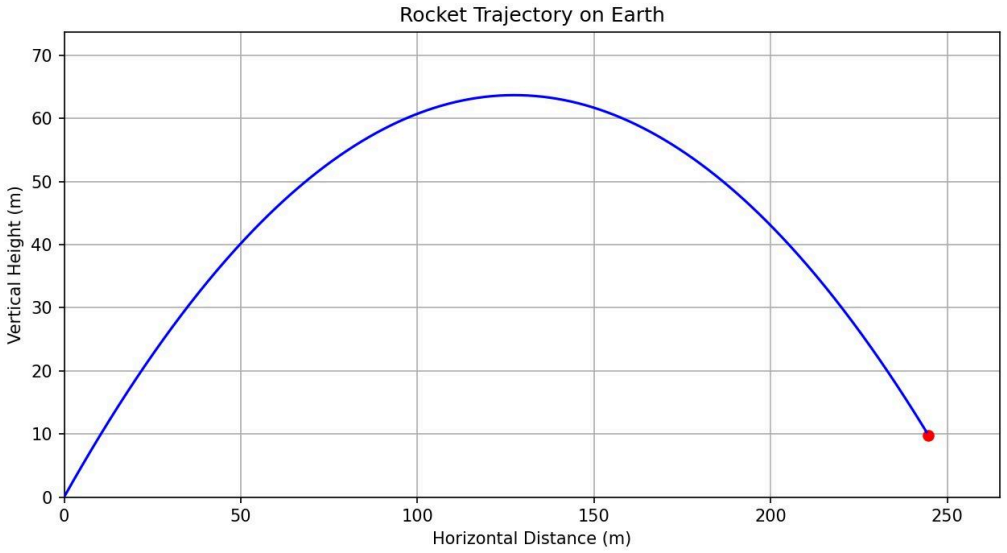
Another major observation was that the launch angle significantly affected the horizontal range and overall trajectory shape. Lower launch angles produced longer horizontal motion with smaller heights, while higher launch angles increased vertical height but reduced horizontal range. Through repeated testing with different angles, it was observed that balanced launch angles produced the most efficient trajectory paths. The graphical trajectory plots and animation outputs helped visualize these observations more clearly. The animated simulation made it easier to compare projectile motion on different planets and understand how changes in gravity directly influence rocket movement.

The results obtained from this project demonstrate how computational simulations can effectively represent real-world physics concepts using programming and mathematical modeling. Additionally, the project highlighted the practical application of Python programming in scientific visualization, simulation systems, and computational analysis.




Rocket Trajectory on Moon
Vertical Height (m)
Horizontal Distance (m)
0
50
100
150
200
250
300
350
0
200
400
600
800
1000
1200
1400


Rocket Trajectory on Mars
Vertical Height (m)
Horizontal Distance (m)
0
20
40
60
80
0
100
200
300
400
500



7. Limitations

Although the rocket trajectory simulator successfully demonstrated the basic principles of projectile motion and computational simulation, the project contains several limitations due to simplified assumptions and beginner-level modeling techniques.

One major limitation of the simulator is that air resistance was not included in the calculations. In real-world rocket motion, air resistance significantly affects speed, trajectory, and stability. However, for simplicity and easier implementation, the simulation assumed ideal projectile motion without atmospheric drag.

Another limitation is that fuel consumption and thrust variation were not considered. Real rockets continuously lose mass as fuel burns during flight, which affects acceleration and trajectory behavior. In this project, the rocket's velocity was assumed to remain based only on the initial launch conditions.

The simulator was also limited to a two-dimensional (2D) environment. The rocket trajectory was calculated only along horizontal and vertical axes, while real aerospace

simulations often involve three-dimensional motion, rotational dynamics, wind conditions, and orbital mechanics.

Additionally, the simulation used simplified projectile motion equations and assumed constant gravitational acceleration throughout the flight. In actual aerospace applications, gravity may vary slightly depending on altitude and environmental conditions.

Despite these limitations, the project was successful in demonstrating how programming and physics can be combined to create computational simulations. The simplified approach also helped make the project easier to understand while building a strong foundation for future improvements and more advanced aerospace simulations.

8. Future Scopes

Although the current simulator successfully demonstrates basic projectile motion and rocket trajectory visualization, there are several areas where the project can be improved and expanded in the future to create a more realistic and advanced aerospace simulation system.

One possible improvement is the inclusion of air resistance and drag forces in the calculations. In real-world rocket motion, atmospheric drag plays a major role in affecting speed, stability, and trajectory. Adding drag equations would make the simulation more accurate and realistic.

Another future enhancement could involve modelling fuel consumption and thrust variation during flight. Real rockets continuously lose mass as fuel burns, which changes acceleration and overall motion. Including these factors would allow the simulator to better represent actual aerospace systems.

The project can also be expanded from a two-dimensional simulation to a three-dimensional (3D) model. A 3D simulation would allow more realistic visualization of rocket movement and could include factors such as wind direction, rotational motion, and spatial orientation.

In the future, orbital mechanics and spaceflight calculations could also be implemented. This would allow the simulator to model satellite launches, planetary orbits, escape velocity, and other advanced aerospace concepts.

Another interesting future improvement would be the integration of artificial intelligence and machine learning techniques for trajectory optimization. AI-assisted systems could automatically determine optimal launch angles, velocities, and flight paths for different planetary environments.

Additionally, real-time physics engines and advanced visualization systems could be integrated to improve simulation quality and interactivity. This would make the simulator more immersive and useful for educational demonstrations, research experiments, and beginner aerospace studies.

Overall, this project provides a foundational framework that can be expanded into a much more advanced computational and aerospace simulation system in the future.

9. Conclusion

The study successfully demonstrated how gravitational variations significantly influence projectile motion and rocket trajectories under different planetary conditions. By using Python programming along with mathematical modelling and visualization techniques, the simulator was able to represent projectile motion on Earth, the Moon, and Mars effectively.

The project also highlighted the practical application of computational simulations in understanding aerospace and physics concepts. Through trajectory analysis, graphical visualization, and animation, the simulator provided a clearer understanding of how launch angle, velocity, and gravity affect rocket motion.

Additionally, the project strengthened understanding of programming concepts such as conditional statements, arrays, functions, mathematical calculations, graph plotting, and animation implementation using Python libraries like NumPy and Matplotlib.

Overall, this research demonstrated how beginner-level computational simulations can be used as an effective tool for learning scientific concepts while also providing a foundation for more advanced aerospace and simulation-based projects in the future.

10. References

1. NASA. *Projectile Motion and Rocket Science Concepts*. Available at: https://www.nasa.gov/
2. NumPy Documentation. *Scientific Computing with Python*. Available at: https://numpy.org/doc/
3. Matplotlib Documentation. *Python Visualization Library*. Available at: https://matplotlib.org/stable/index.html
4. Python Software Foundation. *Python Documentation*. Available at: https://docs.python.org/3/
5. Khan Academy. *Projectile Motion and Two-Dimensional Motion*. Available at: https://www.khanacademy.org/science/physics/two-dimensional-motion
6. HyperPhysics. *Projectile Motion Concepts*. Available at: http://hyperphysics.phy-astr.gsu.edu/
7. OpenStax Physics. *Projectile Motion and Kinematics*. Available at: https://openstax.org/subjects/science
8. Abba, S. (2020). *Modeling Rocket Flight Trajectory*. ResearchGate. Available at: https://www.researchgate.net/
9. Halliday, D., Resnick, R., & Walker, J. *Fundamentals of Physics*. Wiley Publications.
10. Sutton, G. P., & Biblarz, O. *Rocket Propulsion Elements*. Wiley Publications.